TELECOM Paris

IP PARIS

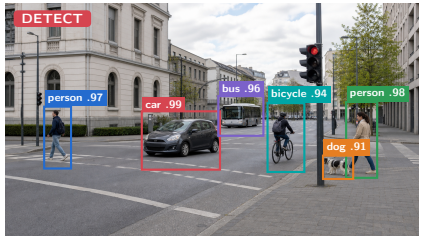
Deep Learning for Computer Vision

From pixels to structured scene understanding

Jhony H. Giraldo (Enseignant–Chercheur)
jhony.giraldo@telecom-paris.fr
13 November 2026

Institut Mines-Télécom

What can machines understand from pixels?



One image; many levels of understanding.

Is computer vision relevant? Why are you enrolled?

Is computer vision relevant? Why are you enrolled?

- Yes! Nowadays, there are many successful applications of computer vision:

Healthcare

Detect disease earlier; quantify anatomy; support clinicians.

Mobility

Understand roads, agents, intent, and risk under uncertainty.

Industry

Inspect defects, monitor processes, and enable flexible automation.

Robotics

Perceive geometry, grasp objects, and act safely in the world.

Sport & media

Track players, recover pose, index video, augment live content.

Earth observation

Map land use, measure change, and respond to hazards at scale.

Is computer vision relevant? Why are you enrolled?

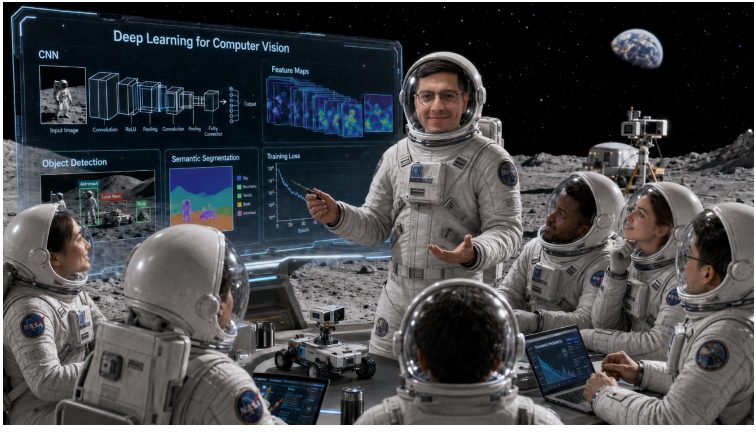
- Yes! Nowadays, there are many successful applications of computer vision:

Healthcare	Mobility	Industry
Detect disease earlier; quantify anatomy; support clinicians.	Understand roads, agents, intent, and risk under uncertainty.	Inspect defects, monitor processes, and enable flexible automation.

Robotics	Sport & media	Earth observation
Perceive geometry, grasp objects, and act safely in the world.	Track players, recover pose, index video, augment live content.	Map land use, measure change, and respond to hazards at scale.

- There's a huge industry in computer vision. There are also researchers working in academia.

Image and Video Generation



Prompt: Generate an image of my students and me in the deep learning for computer vision course on the moon.

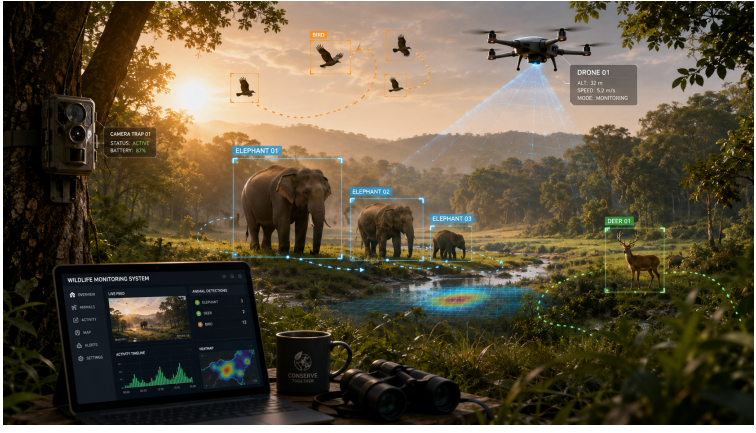
Example of applications

Autonomous Vehicles and Transport

Example of applications

Medical Imaging

Environmental Applications



All images about applications were generated with ChatGPT.

Course objectives

Applications

Choose, train, evaluate, and diagnose modern vision systems.

- Match an output representation to a task.
- Build reproducible PyTorch pipelines.
- Interpret metrics and failure modes.

Fundamentals

Understand why architectures, losses, data, and optimization choices work.

- Connect inductive bias to data efficiency.
- Reason about geometry, uncertainty, and scale.
- Read papers beyond their headline result.

Course objectives

Applications

Choose, train, evaluate, and diagnose modern vision systems.

- Match an output representation to a task.
- Build reproducible PyTorch pipelines.
- Interpret metrics and failure modes.

- CV is not just deep learning.
- Deep learning is not plug-and-play.

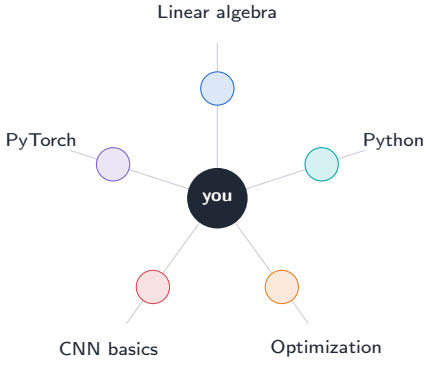
Fundamentals

Understand why architectures, losses, data, and optimization choices work.

- Connect inductive bias to data efficiency.
- Reason about geometry, uncertainty, and scale.
- Read papers beyond their headline result.

Prerequisites

- **Linear algebra**: vectors, matrices, tensors, dot products
- **Optimization**: gradients, SGD, regularization
- **Deep learning**: CNN basics, losses, backpropagation
- **Practice**: Python and basic PyTorch workflows



- **We will refresh some concepts needed today**.

Course calendar

Date	Type	Focus	Responsible
13 Nov 2026	CM	CV recap; CNNs/ViTs; detection, segmentation, pose estimation, tracking	Jhony H. Giraldo
20 Nov 2026	TP	From detection to multi-object tracking	Jhony H. Giraldo
27 Nov 2026	CM	Foundation models; contrastive and vision–language learning	Stephan Alaniz
04 Dec 2026	TP	Foundation-models	Jhony H. Giraldo, Stephan Alaniz
11 Dec 2026	CM	Generative foundation models and language models	Stephan Alaniz
18 Dec 2026	CM	Deep learning for 3D: meshes and point clouds	Jhony H. Giraldo
08 Jan 2027	TP	3D deep learning	Jhony H. Giraldo
15 Jan 2027	Eval.	Oral paper discussion / course evaluation	Course team

All sessions are scheduled in the afternoon. Some students will be evaluated on the morning of January 15. CM = cours magistral; TP = travail pratique.

Evaluation

Continuous assessment — 50%

- Two *one-hour written tests*, held during the second half of:
  - *TP 2, December 04*: CM 1 + TP 1 + CM 2.
  - *TP 3, January 08*: TP 2 + CM 3 + CM 4.
- Tests cover both **lecture material and practical sessions**.

Final project and oral exam — 50%

- Work in groups of two on an approved recent research paper.
- Present the problem, method, main results, and limitations.
- Formulate a testable hypothesis inspired by the paper or one of its limitations.
- Keep the hypothesis **feasible** given your time and computational resources.
- Test the hypothesis experimentally and provide evidence for your conclusions.
- Submit a working and reproducible notebook before the final presentation.
- Answer individual questions during the final presentation.

Use of LLMs/agents: You may use them for the final project, but you remain responsible for all presented material and must be able to explain and defend every choice. **We strongly recommend developing the research hypothesis yourselves.**

Expectations and policies

- Limit the use of cell phones during lectures for non-academic purposes.
- Participation in the lectures is important.
- Academic integrity: plagiarism, cheating, and misuse of LLMs will be penalized. You should verify every claim and run every submitted result.
- Examples of misuse of LLMs:
  - Submit/present something you cannot explain.
  - Use invented references, data, or measurements.

You remain responsible for every line, claim, figure, and number you submit.

Resources

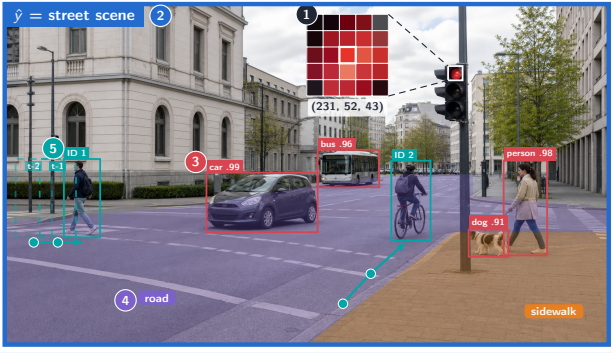
Concepts

- Stanford CS231n: cs231n.stanford.edu
- Goodfellow, Bengio, Courville: Deep Learning
- Original papers: read the method *and* the ablations

Code and empirical context

- torchvision
- Detectron2
- MMDetection / MMCV

Today: beyond a single label



1 pixels $x \in [0, 255]^{H \times W \times 3}$
raw measurements, no meaning

2 class $y \in \{1, \ldots, C\}$
what is the scene? (no *where*)

3 boxes $\{(b_k, c_k, s_k)\}_{k=1}^{K}$
what + where, for every object

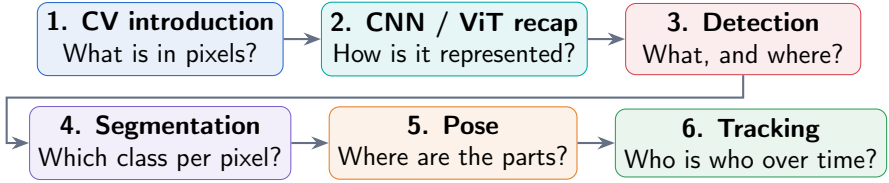
4 masks $M \in \{1, \ldots, C\}^{H \times W}$
which class at every pixel?

5 tracks $\{(t, b_t^i, \mathrm{ID}_i)\}$
who is who, over time?

more structure

From 2 to 5: increasingly **structured** outputs—and costlier supervision.

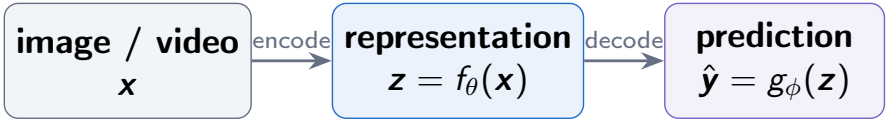
Outline



- The backbone is reusable; the representation, head, loss, metric, and inference must match the task.

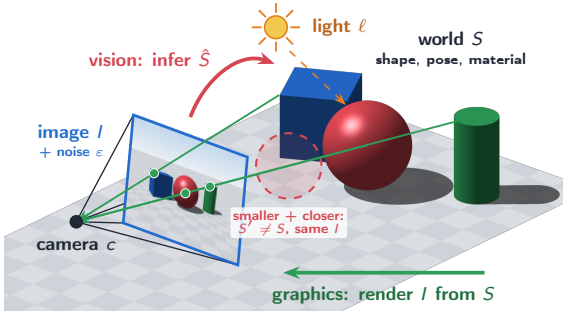
Computer vision recap

From pixels to decisions



- $f_\theta(\cdot)$ is typically a neural network (convolution-based, attention-based, etc.), parametrized by the parameters θ; while $g_\phi(\cdot)$ is another neural network, typically called the prediction head, parametrized by the parameters ϕ.
- **Learning question:** What representation makes the target predictable from finite data?

What is computer vision? An inverse problem



Ill-posed

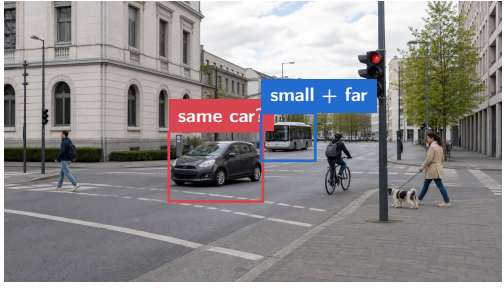
Many worlds can produce nearly the same image:

$$I = R(S, \ell, c) + \varepsilon,$$

with lighting ℓ, camera c, and noise or unmodeled effects ε.

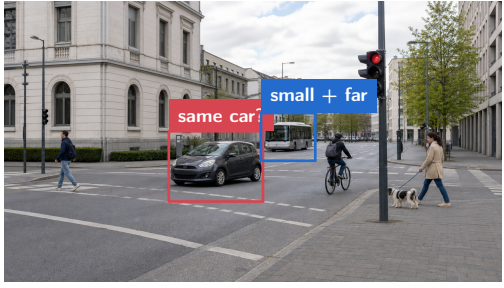
Vision succeeds by combining data with priors.

Why is computer vision hard?



- **Projection**: depth collapses onto the image plane.
- **Appearance**: lighting, weather, texture, sensor noise.
- **Geometry**: viewpoint, deformation, articulation, scale.
- **Visibility**: occlusion, truncation, clutter.
- **Semantics**: context and ambiguous categories.
- **Open world**: rare and unseen cases matter.

Why is computer vision hard?



- **Projection**: depth collapses onto the image plane.
- **Appearance**: lighting, weather, texture, sensor noise.
- **Geometry**: viewpoint, deformation, articulation, scale.
- **Visibility**: occlusion, truncation, clutter.
- **Semantics**: context and ambiguous categories.
- **Open world**: rare and unseen cases matter.

- Useful invariance must coexist with sensitivity to location, shape, and identity.

Computer-vision task taxonomy

Task	Output unit	Central question	Typical metric
Classification	image	What is present?	top-k accuracy
Localization	one box + class	Where is the main object?	IoU / accuracy
Detection	set of boxes	What objects, and where?	AP / mAP
Semantic segmentation	pixel	Which class per pixel?	mIoU
Instance segmentation	object mask	Which pixels belong together?	mask AP
Pose estimation	keypoint / part	Where are articulated parts?	PCK / OKS AP
Tracking	trajectory + ID	Which observation is the same entity?	HOTA / IDF1

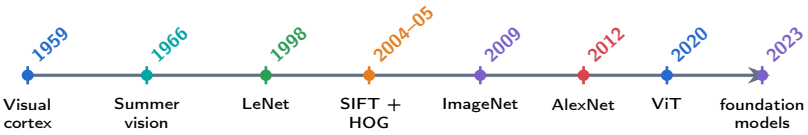
The output space determines annotation cost, architecture, loss, inference, and metric.

One image, progressively richer annotations

image label | boxes + classes | dense class map | keypoints + limbs | identities over time

more spatial structure, more annotation, more ways to be wrong

A brief history on computer vision



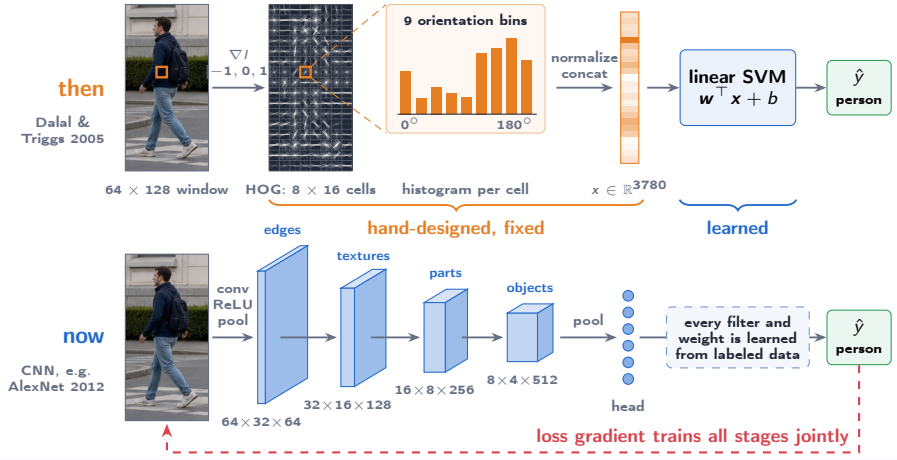
hand-designed features
geometry + invariance

supervised deep learning
data + accelerators

pretraining at scale
transferable representations

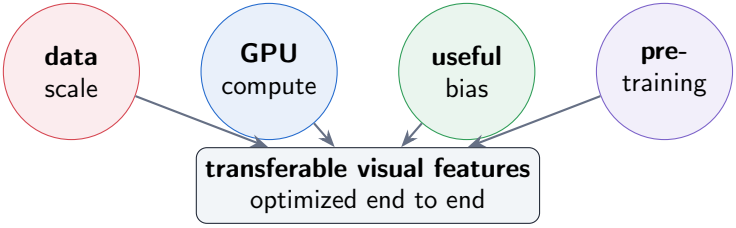
Milestones: Hubel & Wiesel (1959/1962); MIT Summer Vision Project (1966); LeCun et al. (1998); Lowe (2004); Dalal & Triggs (2005); Deng et al. (2009); Krizhevsky et al. (2012); Dosovitskiy et al. (2021).

From handcrafted features to learned representations



End-to-end learning replaces hand-designed interfaces, but not the need for good assumptions or evaluation.

Why deep learning changed computer vision



- Convolution encoded locality; modern attention adds adaptive global interaction.
- Large labeled datasets made comparisons credible; large unlabeled datasets widened pretraining.
- Better optimization, normalization, and residual connections made much deeper networks trainable.

Reminder: supervised learning setup

$$\mathcal{D}_{\text{train}} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N}, \qquad \hat{\boldsymbol{y}}_i = f_\theta(\boldsymbol{x}_i)$$

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\theta(\boldsymbol{x}_i), \boldsymbol{y}_i) + \lambda\, \Omega(\theta).$$

The loss is optimized on training data; choices are made on validation data; the test set estimates final generalization.

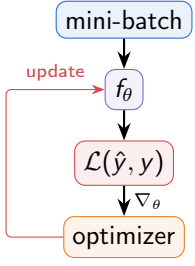


Image classification

$$\boldsymbol{s} = f_\theta(\boldsymbol{x}) \in \mathbb{R}^C,$$

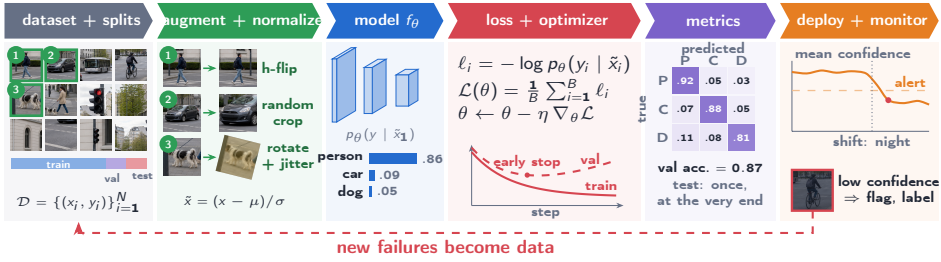
$$p(y = c \mid \boldsymbol{x}) = \frac{e^{\boldsymbol{s}_c}}{\sum_{k=1}^{C} e^{\boldsymbol{s}_k}}.$$

class	probability
street scene	**0.84**
parking lot	0.09
bus station	0.05
other	0.02

Cross-entropy rewards probability on the ground-truth class:

$$\mathcal{L}_{\mathrm{CE}} = -\log p(y \mid \boldsymbol{x}).$$

A machine-learning pipeline for images



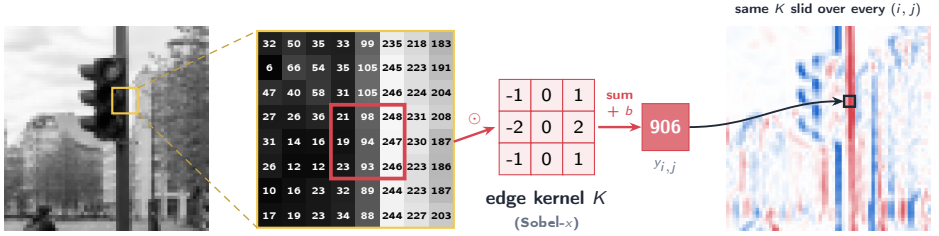
Before training

Define the target, split, inspect labels, and establish a simple baseline.

Common leakage

The test/validation data are not truly independent of the training data, so the measured performance looks better than the model's real ability to generalize

Convolution intuition: a learned local pattern detector



$$y_{i,j} = b + \sum_{u,v,c} K_{u,v,c}\, x_{i+u,j+v,c}$$

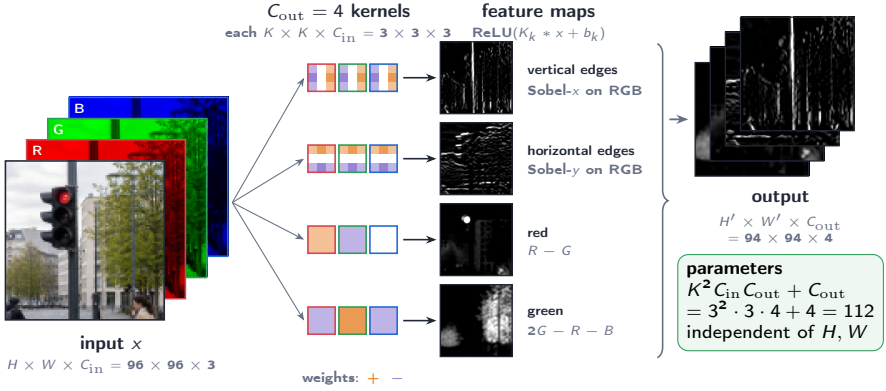
red window ($b = 0$):

$$-21 + 248 - 2 \cdot 19 + 2 \cdot 247 - 23 + 246 = 906$$

- Same weights at every location.
- High response = local pattern match.
- Training learns K; sliding maps where the pattern occurs.

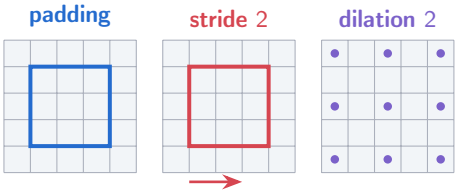
Convolution is a local linear operator with shared parameters.

Convolution layer: channels, kernels, feature maps



- Each output channel mixes *all* input channels but only a local spatial neighborhood.
- A 1×1 convolution changes channels without enlarging the spatial receptive field.

Stride, padding, and dilation



For one spatial dimension,

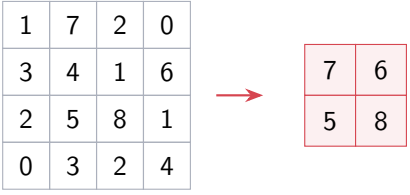
$$H' = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} + 1 \right\rfloor .$$

- S: sampling step / downsampling
- P: border context and output size
- D: spacing inside the kernel
- effective kernel: $D(K-1)+1$

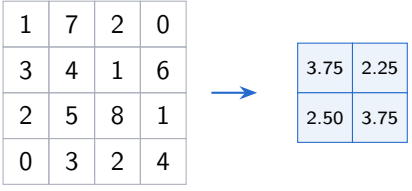
These hyperparameters determine resolution, compute, and visible context.

Pooling: invariance versus spatial precision

Max pool 2×2

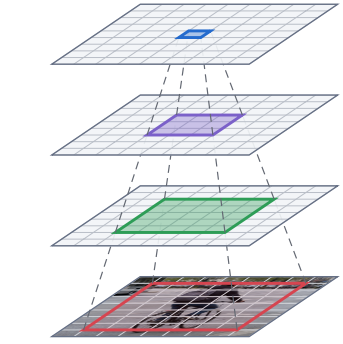


Average pool 2×2



- Pooling summarizes a neighborhood and reduces resolution and compute.
- It increases tolerance to small shifts, but exact boundaries and keypoints become harder to recover.
- Modern networks often downsample with a strided convolution instead.

Receptive field: how much context can a feature use?



each step up: 3×3 conv, stride 1

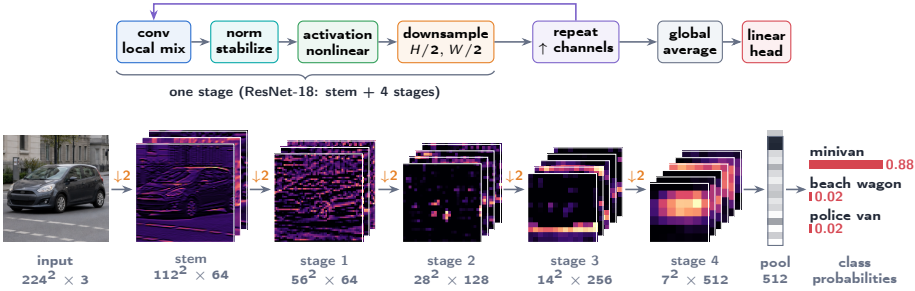
Track the sampling jump j_l and receptive field r_l:

$$j_l = j_{l-1}\, s_l,$$
$$r_l = r_{l-1} + (k_l - 1)\, d_l\, j_{l-1}.$$

Left: $k_l = 3$, $s_l = d_l = 1 \Rightarrow r_l = 1 + 2l$.
With stride $s_2 = 2$: $j_2 = 2$, so
$r_3 = 5 + 2 \cdot 2 = 9$.

- Depth, stride, and dilation expand context.
- The *theoretical* field is not equal to the influence distribution.
- Dense prediction needs both broad context and high-resolution detail.

The recurring CNN architecture pattern



Spatial trade

H, W decrease while receptive field and semantic abstraction increase.

Channel trade

More feature channels preserve diverse evidence as spatial resolution shrinks.

- A classifier discards spatial detail at the end; dense tasks tap intermediate maps before it disappears.

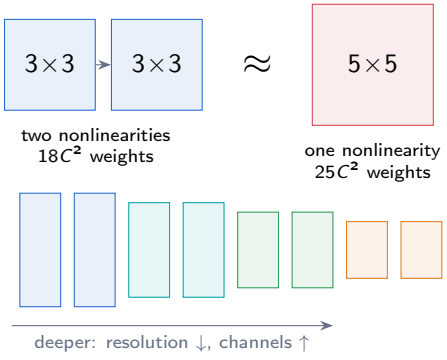
LeNet to AlexNet: depth meets data and compute

	LeNet-5 (1998)	AlexNet (2012)	Why the jump mattered
Input	32 × 32 grayscale	224 × 224 RGB	richer natural-image variation
Feature extractor	2 conv stages	5 conv stages	deeper hierarchy, larger capacity
Nonlinearity	tanh / sigmoid	ReLU	faster, less saturating optimization
Regularization	subsampling	dropout + augmentation	control overfitting at scale
Compute	CPU-era design	2 GPUs	feasible large-scale convolution
Benchmark	digit recognition	ImageNet	decisive natural-image result

AlexNet's impact came from a system of choices—architecture, data, compute, augmentation, and optimization.

LeCun et al., "Gradient-Based Learning Applied to Document Recognition," Proc. IEEE, 1998. Krizhevsky, Sutskever & Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," NeurIPS, 2012.

VGG: small filters, deep and simple

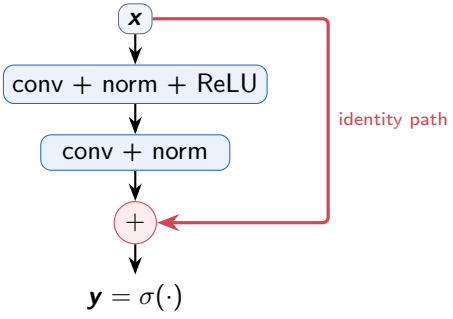


- Uniform 3×3 convolutions.
- 16 or 19 weight layers.
- Easy to reason about and reuse.
- Heavy classifier: VGG-16 has about 138M parameters.

Its clean design became a template, even though later networks are more compute- and parameter-efficient.

Simonyan & Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," ICLR, 2015.

ResNet: learn a residual correction



$$\boldsymbol{y} = \sigma(\mathcal{F}(\boldsymbol{x}; \theta) + \boldsymbol{x}) .$$

- Identity paths carry signal and gradients.
- A block can learn a small correction around identity.
- Projection shortcuts align changed shapes.
- Very deep networks become trainable and reusable.

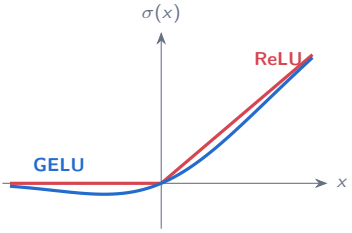
He, Zhang, Ren & Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016.

Normalization and activations

Batch normalization (per channel)

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta.$$

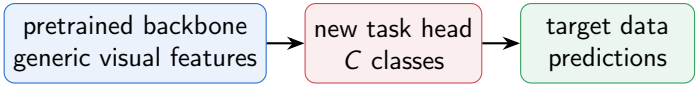
It improves optimization and permits larger learning rates, but behavior depends on batch statistics and train/eval mode.



- ReLU: cheap, sparse, piecewise linear.
- GELU: smooth gating, common in transformers.
- LayerNorm normalizes within a token/example and does not need batch statistics.

Ioffe & Szegedy, "Batch Normalization," ICML, 2015. Hendrycks & Gimpel, "Gaussian Error Linear Units," 2016.

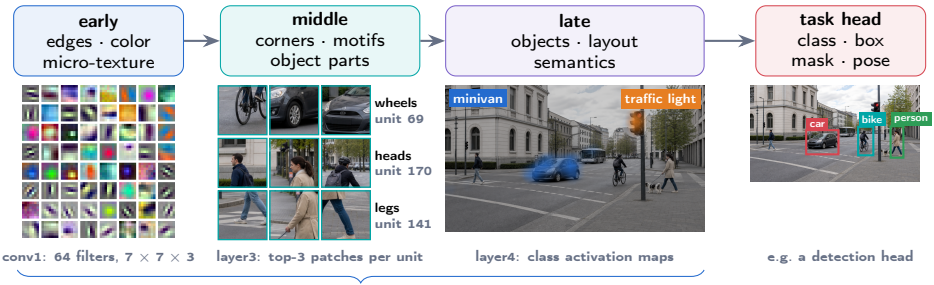
Transfer learning: reuse a pretrained backbone



Strategy	Trainable weights	Best when	Main risk
Linear probe	head only	little data; diagnose representation	under-adaptation
Partial fine-tune	head + late blocks	moderate data / related domain	brittle layer choice
Full fine-tune	all weights	enough data and compute	overfit / forgetting

Always adapt preprocessing and normalization to the exact pretrained checkpoint; validation determines the strategy.

A hierarchy of visual features



- Classification can use the final semantic vector.
- Detection and segmentation combine late semantics with earlier spatial detail.
- "Features" are task- and training-dependent, not a universal visual vocabulary.

CNN strengths: useful inductive bias

Translation equivariance

If T_δ shifts the image, convolution shifts the feature map:

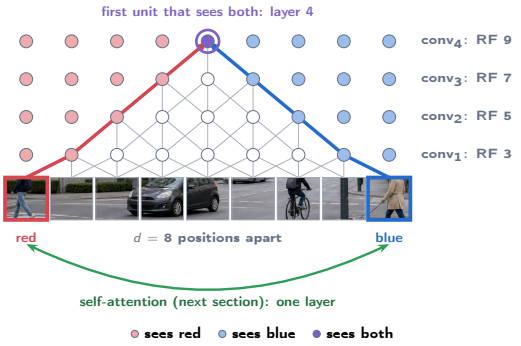
$$K * (T_\delta x) = T_\delta (K * x).$$

Equivariance preserves *where*; pooling can later create partial invariance.

- **Locality**: nearby pixels interact first.
- **Parameter sharing**: the same detector works across locations.
- **Hierarchical composition**: local evidence builds into parts and objects.
- **Data efficiency**: built-in assumptions constrain the hypothesis space.
- **Multi-scale maps**: intermediate layers naturally support dense tasks.

- A good inductive bias says which solutions should be easy to learn.

CNN limitations: global context is indirect



$k \times k$ convs: layers needed $\approx d/(k-1)$, here $8/2 = 4$

- Long-range interaction needs depth, large kernels, dilation, or pooling.
- Repeated downsampling discards fine spatial detail.
- Fixed local mixing does not adapt its connectivity to the image.
- Strong locality can favor texture or shortcuts over global shape.

These are tradeoffs, not a verdict: modern CNNs can be very strong, and transformers introduce their own costs.

Attention intuition: content-dependent interaction

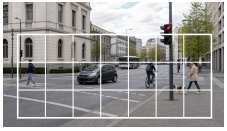
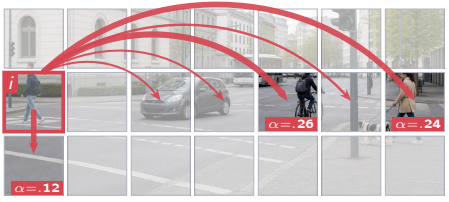


image → 7 × 3 patches
each patch becomes a token
(shown separated below)

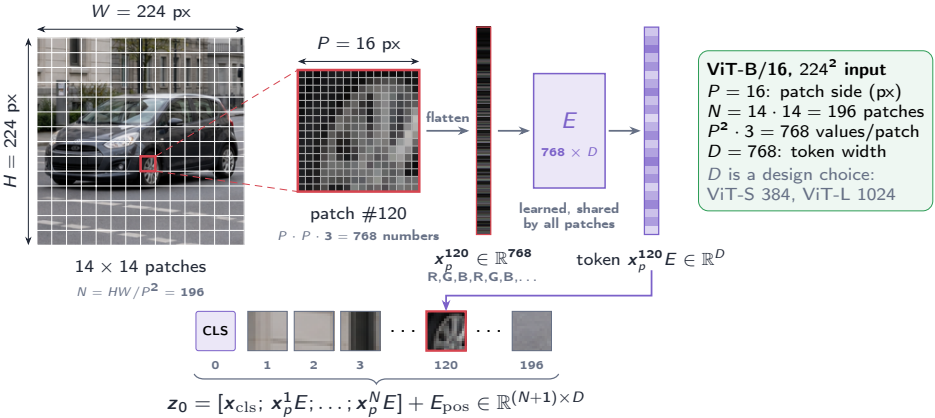


query patch i attends most to the other people, wherever they are

$$z'_i = \sum_j \alpha_{ij} v_j, \qquad \sum_j \alpha_{ij} = 1.$$

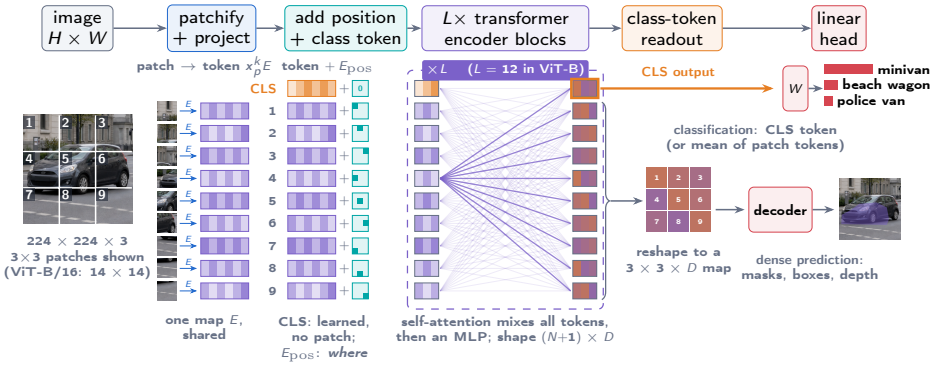
- Weights α_{ij} depend on token content.
- Interaction is global in a standard transformer.
- Geometry must be supplied through position information.
- Full attention costs $\mathcal{O}(N^2)$ token interactions.

From an image to a sequence of patches



- Smaller patches preserve detail but increase N and attention cost.
- The class token is a learnable global readout; alternatives include average pooling.

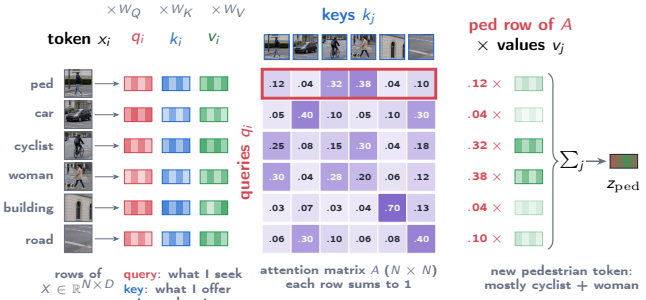
Vision Transformer pipeline



Dosovitskiy et al., "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale," ICLR, 2021.

Self-attention mechanics: query, key, value

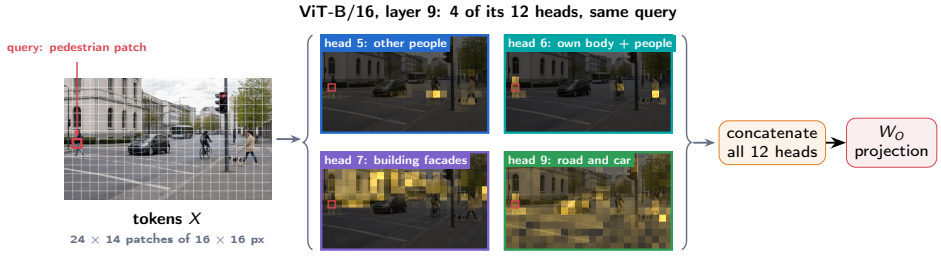
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad A = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big), \quad \mathrm{Attn}(X) = AV$$



- $q_i \cdot k_j$: compatibility of every (query, key) pair: $QK^\top$ is $N \times N$.
- $1/\sqrt{d_k}$: keeps the scores at a workable scale.
- Row-wise softmax: each query's weights are positive and sum to 1.
- Output row i = weighted mix of the values v_j.

Weights here are illustrative.

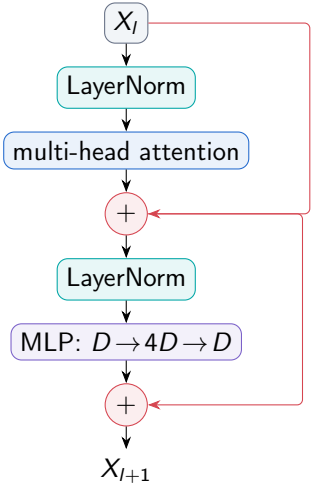
Multi-head attention: several relational views



$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h) W_O.$$

Heads provide separate learned subspaces, not guaranteed human-interpretable roles.

Transformer block: mix tokens, then mix channels



Two complementary operations

Attention mixes information *between tokens*.

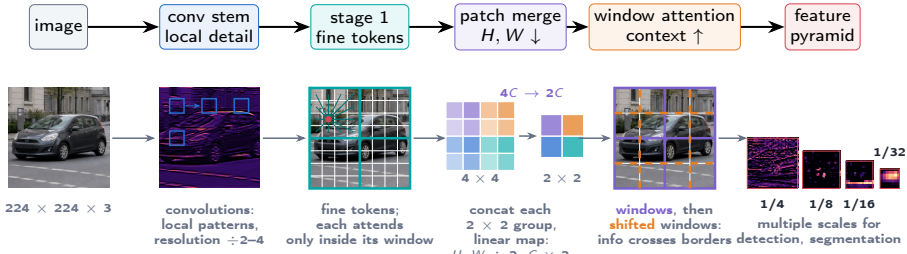
MLP mixes information *between channels* independently at each token.

- Residual paths stabilize deep optimization.
- Pre-normalization is common in modern variants.
- Position information prevents permutation ambiguity.

CNN versus ViT: different inductive biases

Dimension	Convolutional network	Vision Transformer
Connectivity	local, fixed kernel support	global and content-dependent by default
Position	encoded by grid and convolution	added explicitly to patch tokens
Weight sharing	across spatial locations	projections shared across tokens
Translation bias	strong equivariance before boundaries/stride	weaker; learned from data/augmentation
Data regime	often strong with less data	benefits greatly from large-scale pretraining
Resolution cost	roughly linear in pixels per conv	full attention quadratic in token count
Dense outputs	natural multi-scale feature maps	tokens must be reshaped / made hierarchical

Hybrid and hierarchical vision models



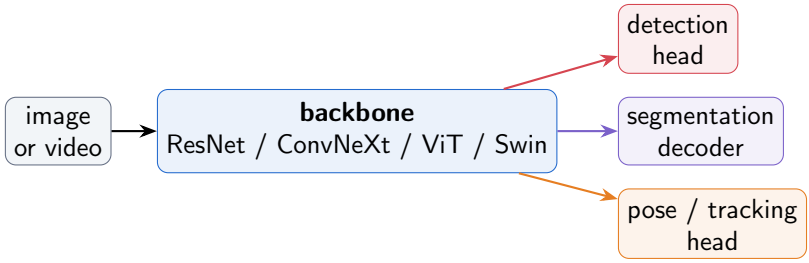
Conv stem
Efficiently captures local patterns and reduces resolution.

Local windows
Reduce attention cost; shifted windows exchange information.

Hierarchy
Produces multiple scales for detection and segmentation.

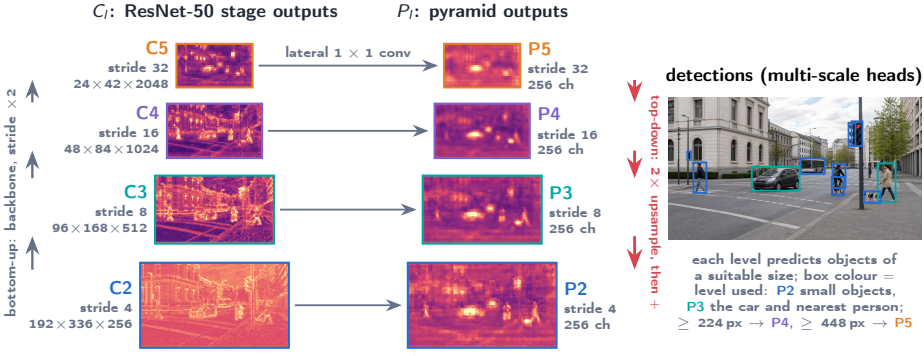
Liu et al., "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," ICCV, 2021.

The backbone: a reusable feature extractor



- A backbone outputs feature vectors or maps; the head maps them to a task representation.
- Pretraining can be supervised, self-supervised, or multimodal.
- "Same backbone" does not imply same preprocessing, scale, output stride, or fine-tuning recipe.

Feature pyramids: a preview of multi-scale prediction



C_l: backbone output after stage l (stride 2^l). P_l: same stride, 256 channels, enriched top-down:

$$P_5 = \mathrm{conv}_{1\times1}(C_5), \qquad P_l = \mathrm{conv}_{3\times3}\big(\mathrm{conv}_{1\times1}(C_l) + \mathrm{up}_{2\times}(P_{l+1})\big)$$

Lin et al., "Feature Pyramid Networks for Object Detection," CVPR, 2017.

Mini-quiz: what controls output size?

Question: A 1D input has length $H = 32$. A convolution uses $K = 3$, stride $S = 2$, padding $P = 1$, and dilation $D = 1$. What is H'?

Question: Which single change below increases the output length: $S : 2 \to 1$, $P : 1 \to 0$, $K : 3 \to 5$, or $D : 1 \to 2$?

Do it without code

$$H' = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} + 1 \right\rfloor.$$

Then explain what each change does to sampling density and effective receptive field.

Mini-quiz answer

$$H' = \left\lfloor \frac{32 + 2(1) - 1(3-1) - 1}{2} + 1 \right\rfloor = \lfloor 16.5 \rfloor = 16.$$

- $S : 2 \to 1$ gives $H' = 32$: **larger output**.
- $P : 1 \to 0$ gives $H' = 15$: smaller.
- $K : 3 \to 5$ gives $H' = 15$: smaller.
- $D : 1 \to 2$ gives $H' = 15$: smaller.

Subtle point

A larger kernel or dilation expands context but does *not* preserve size unless padding is adjusted.

Shape arithmetic is part of architecture design, especially for skip connections and dense outputs.

Classification is not enough: where is the object?

A single label discards

- object count,
- object location and scale,
- overlap and relationships,
- uncertainty per object.

image $\longrightarrow$ **set of objects**

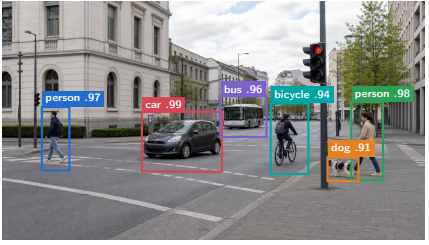
Object detection changes both the prediction space and the evaluation protocol.

Object detection

Outline checkpoint

FROM FEATURES TO OBJECTS

1. Pixels, CNNs, and visual features
2. **Object detection**
3. Segmentation and dense prediction
4. Pose and tracking



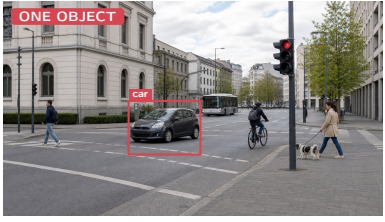
What? class **Where?** box **How sure?** score

- Detection turns one image into a variable-size set of localized objects.

Object localization vs detection

Localization

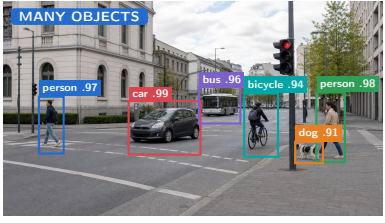
One known object category; predict one box.



$$f_\theta(I) = (p,\, b), \qquad b \in \mathbb{R}^4.$$

Detection

Unknown categories and unknown object count.

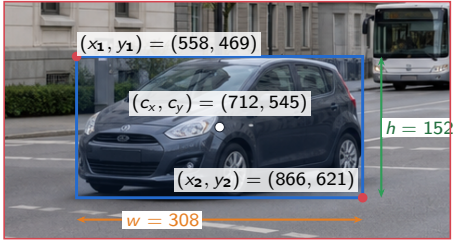


$$f_\theta(I) = \{(p_i,\, b_i)\}_{i=1}^{N}.$$

Detection adds a cardinality problem: the model must decide which candidates correspond to real objects.

Bounding box parameterization

image coordinates:
origin top-left, y **points** *down*
W = **1672**, H = **941 px**



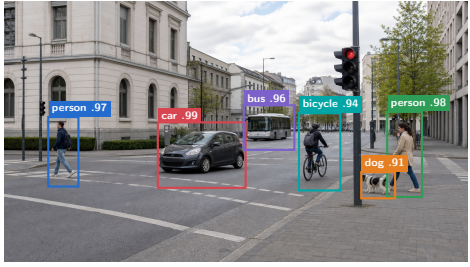
Two equivalent forms

$(x_1, y_1, x_2, y_2) = (558, 469, 866, 621)$
$(c_x, c_y, w, h) = (712, 545, 308, 152)$
$\div(W, H)$: $(0.426, 0.579, 0.184, 0.162)$

- Normalize coordinates by image width and height.
- Enforce $w, h > 0$ through a suitable transform.
- Relative offsets are easier to learn across scales.

A box is only four numbers, but their scale and parameterization shape optimization.

Detection output



$$\hat{\mathcal{Y}} = \{(\hat{p}_i, \hat{b}_i)\}_{i=1}^{K}$$

class probabilities $\hat{p}_i \in [0,1]^{C+1}$

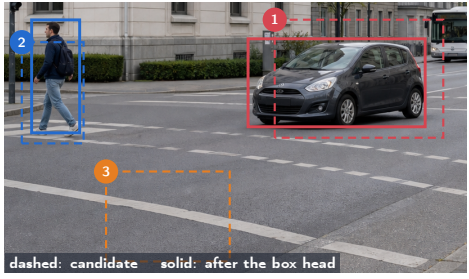
box coordinates $\hat{b}_i \in \mathbb{R}^4$

confidence or "no object"

- The target set has variable size N.
- Networks usually emit a fixed budget $K \gg N$.
- Assignment and filtering select detections.

- Prediction is easy to tensorize; deciding which predictions count is the central difficulty.

Detection as multitask learning



cand.	IoU	class head	box head	loss
1	0.58	car 1.00	refined (solid)	$\mathcal{L}_{\text{cls}} + \lambda \mathcal{L}_{\text{box}}$
2	0.64	person 1.00	refined (solid)	$\mathcal{L}_{\text{cls}} + \lambda \mathcal{L}_{\text{box}}$
3	0.00	background 1.00	not trained	$\mathcal{L}_{\text{cls}}$ only

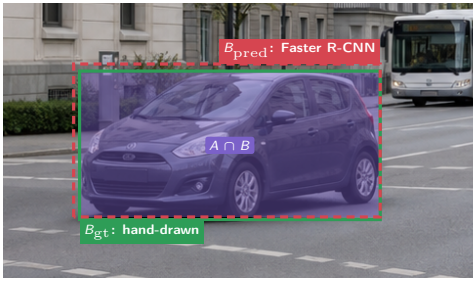
Girshick, Fast R-CNN, 2015; Rezatofighi et al., GIoU, 2019.

Per-candidate objective

$$\mathcal{L}_i = \mathcal{L}_{\text{cls}}(p_i, c_i) + \lambda \, 1[c_i \neq \varnothing] \mathcal{L}_{\text{box}}(b_i, b_i^*).$$

- Every candidate learns foreground/background.
- Only matched positives receive box supervision.
- Loss scales must be balanced.

Intersection over Union (IoU)



IoU $= 0.18$ IoU $= 0.56$ IoU $= 0.89$

dashed: B_{gt} shifted / scaled by hand

Everingham et al., PASCAL VOC, IJCV 2010.

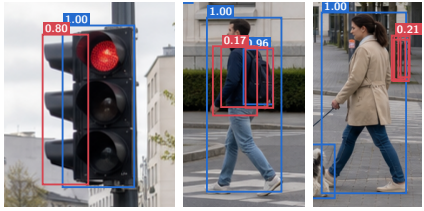
$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1].$$

for the car: $\frac{45\,979 \text{ px}^2}{50\,011 \text{ px}^2} = 0.92$

0: disjoint 1: identical

- Scale invariant.
- Used for matching and evaluation.
- A threshold turns overlap into a discrete TP/FP decision.

Precision and recall for detection



TP: matched, class correct
FP: duplicate (.80), wrong class, no object (.21)

score ≥	TP	FP	FN	precision	recall
0.995	7	0	4	1.00	0.64
0.9	11	0	0	1.00	1.00
0.5	11	1	0	0.92	1.00
0.05	11	10	0	0.52	1.00

11 GT objects; objects < 30 px tall ignored (VOC "difficult")

Match before counting

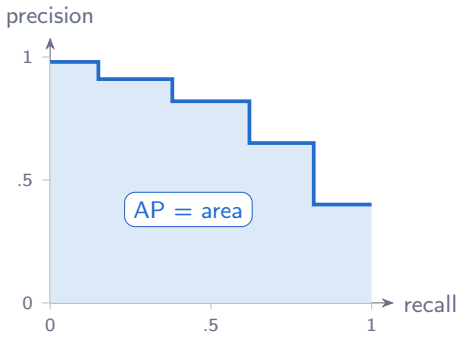
A prediction is a TP when its class is correct, its IoU $\geq \tau$, and its ground-truth object is not already matched.

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad \mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

- Raise the score threshold: usually precision ↑, recall ↓.
- Duplicate boxes count as false positives.
- Missed objects reduce recall.

A detector is evaluated over a confidence sweep, not at one arbitrary threshold.

Average Precision and mAP



Everingham et al., VOC, 2010; Lin et al., COCO, 2014.

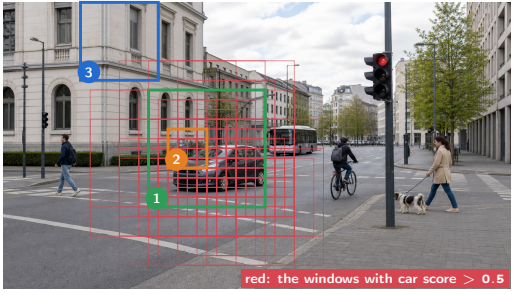
Summarize the PR curve

$$\mathrm{AP}_c = \int_0^1 p_c(r)\, dr$$

$$\mathrm{mAP} = \frac{1}{C} \sum_{c=1}^{C} \mathrm{AP}_c$$

- Sort predictions by confidence.
- Accumulate TP/FP to trace a PR curve.
- COCO averages AP across IoU thresholds .50:.05:.95.

Why not slide a classifier everywhere?



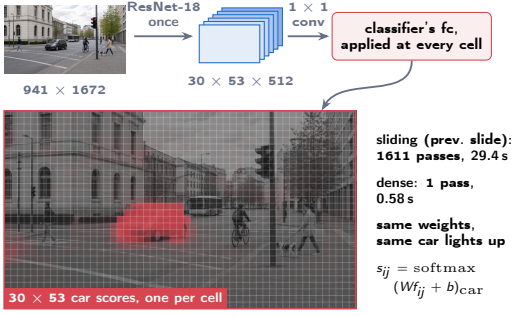
1611 windows (3 sizes)
= 1611 ResNet-18 passes
= 29.4 s on this CPU

116 fire on *one* car
1383 are background ($< .1$)

- Adjacent windows repeat almost identical computation.
- Object size and aspect ratio are unknown.
- One object activates many overlapping windows.
- Background windows vastly outnumber objects.

Question: Which computation can be shared across all windows?

Fully convolutional detection



sliding (prev. slide): **1611 passes**, 29.4 s

dense: **1 pass**, 0.58 s

same weights, same car lights up

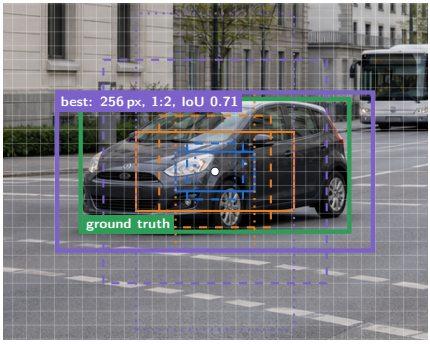
$$s_{ij} = \mathrm{softmax}\left(W f_{ij} + b\right)_{\mathrm{car}}$$

- Run the backbone once.
- Reuse every feature for neighboring predictions.
- Preserve spatial layout in a dense output tensor.
- Increase receptive field without cropping patches.

A sliding classifier becomes one shared convolution over a feature map.

Sermanet et al., OverFeat, ICLR 2014.

Anchor boxes



64 128 256 px; $h{:}w$ = 1:2 solid, 1:1 dashed, 2:1 dotted
grid: stride 16 px; this cell owns 9 anchors

whole image: 104 × 59 cells × 9 = 55224 anchors

Reference geometry

Each location owns boxes with several scales and aspect ratios:
$a = (c_x^a, c_y^a, w_a, h_a)$.

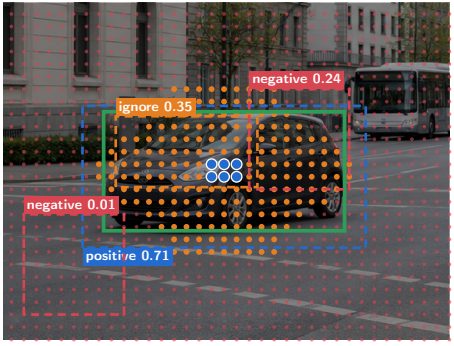
$$\hat{c}_x = c_x^a + t_x\, w_a, \qquad \hat{c}_y = c_y^a + t_y\, h_a,$$
$$\hat{w} = w_a e^{t_w}, \qquad \hat{h} = h_a e^{t_h}.$$

The head predicts objectness, class, and offsets (t_x, t_y, t_w, t_h).

Best anchor $a = (712, 552, 362, 181)$
targets $t = (0.000, -0.039, -0.162, -0.175)$
$\Rightarrow \hat{b} =$ the car, $(712, 545, 308, 152)$

Ren et al., Faster R-CNN, NeurIPS 2015; Liu et al., SSD, ECCV 2016.

Anchor matching



dot = one cell, coloured by its best anchor: IoU $\geq$ .7 .3–.7 $<$ .3

whole image, 11 objects: 55224 anchors
$\rightarrow$ 22 positive, 1607 ignored, 53595 negative

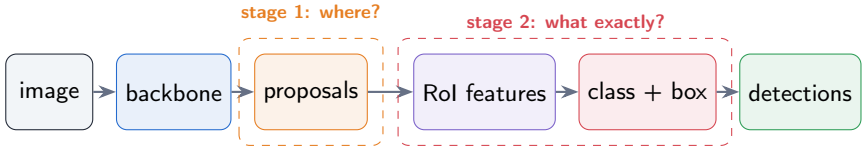
$$y(a,g) = \begin{cases} +1 & \text{IoU}(a,g) \geq \tau_+, \\ 0 & \text{IoU}(a,g) < \tau_-, \\ \perp & \text{otherwise.} \end{cases}$$

Faster R-CNN RPN: $\tau_+ = 0.7$, $\tau_- = 0.3$

- Thresholds are design choices, not universal constants.
- Ignored ($\perp$) anchors contribute no classification loss.
- Assignment defines the labels seen during training.

Ren et al., Faster R-CNN, 2015.

Two-stage detectors: proposals, then decisions



Strength

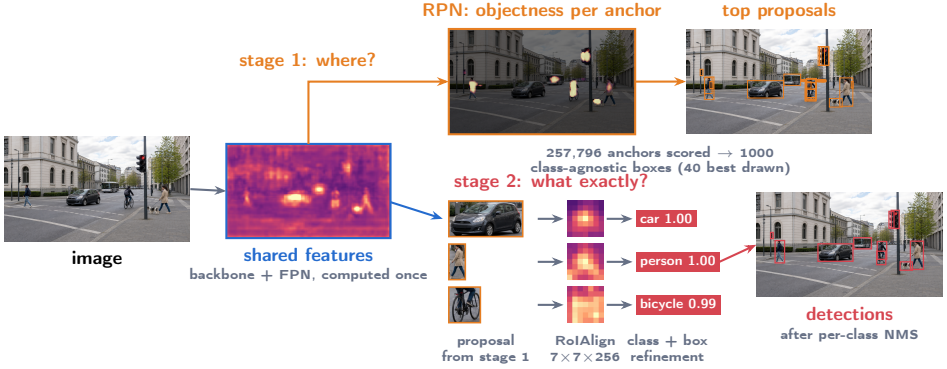
Expensive per-object reasoning is applied only to a sparse proposal set.

Cost

Proposal generation and per-RoI processing add latency and pipeline complexity.

Girshick et al., R-CNN, 2014; Girshick, Fast R-CNN, 2015.

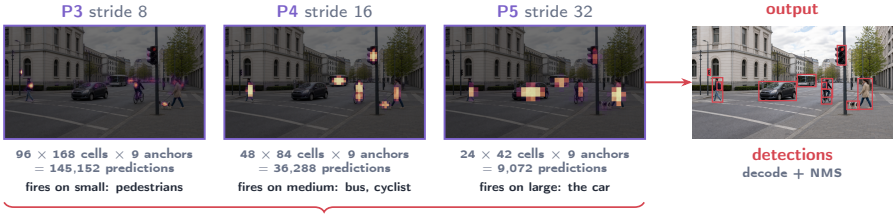
Faster R-CNN



- The Region Proposal Network is learned.
- RPN and detector reuse the same feature map.
- Proposals are class agnostic.
- The second stage classifies and refines each RoI.

Ren, He, Girshick, and Sun, Faster R-CNN, NeurIPS 2015.

One-stage detectors



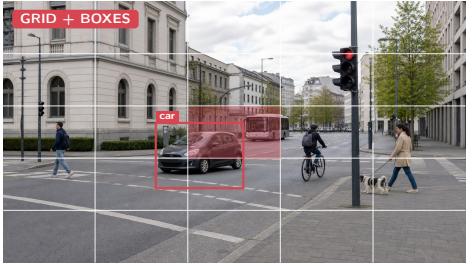
One pass

No explicit proposal/RoI stage: predict densely at several resolutions.

- Low latency and simple batching.
- Multi-scale maps serve different object sizes.
- Dense supervision creates severe class imbalance.

Liu et al., SSD, ECCV 2016; Lin et al., FPN, CVPR 2017; Lin et al., RetinaNet, ICCV 2017.

YOLO intuition



Historical YOLOv1 view

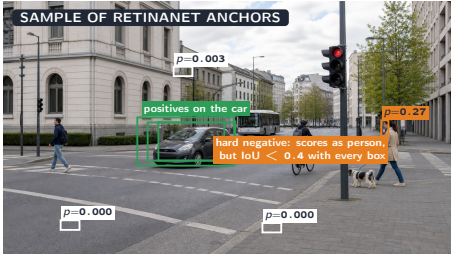
The grid cell containing an object's center is responsible for predicting it.

$$S \times S \times (B \cdot 5 + C)$$

- One network maps pixels directly to detections.
- Joint reasoning uses global image context.
- Modern variants use different heads and assignments.

Redmon et al., YOLO, CVPR 2016.

Class imbalance and focal loss

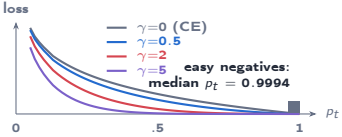


	anchors	share of total loss: CE	focal
positives	288	3.6%	72.0%
hard negatives	1,454	12.1%	27.2%
easy negatives	191,102	84.2%	0.9%

Focal loss

$$\mathrm{FL}(p_t) = -\alpha_t(1-p_t)^{\gamma}\log p_t$$

The modulating factor suppresses gradients from already-correct examples.



- $\gamma = 0$ recovers weighted cross-entropy.
- Larger γ focuses learning on hard mistakes.
- It enabled accurate dense one-stage detection in RetinaNet.

Lin et al., Focal Loss, ICCV 2017. Shares: RetinaNet (α=.25, γ=2) on this scene.

Non-maximum suppression

rank	score	IoU with #1	fate
1	0.96	—	kept
2	0.95	0.99	suppressed ($> \tau$)
3	0.95	0.98	suppressed ($> \tau$)
20	0.82	0.98	suppressed ($> \tau$)
40	0.43	0.95	suppressed ($> \tau$)
59	0.19	0.85	suppressed ($> \tau$)
79	0.06	0.80	suppressed ($> \tau$)

Whole image, all classes: **3,394** raw boxes → **636** after NMS.

1. Select the highest-score box.
2. Remove same-class boxes whose IoU with it exceeds τ_{nms}.
3. Repeat until no candidates remain.

NMS converts redundant local votes into one detection per object.

Neubeck and Van Gool, Efficient NMS, ICPR 2006. Candidates: RetinaNet (COCO) on this scene.

NMS limitations

Overlapping real objects

IoU $= 0.31$ (shaded)

per-class NMS: both kept ✓

class-agnostic, τ=0.3: **bicycle deleted** ×

Soft-NMS: bicycle 0.99 $\rightarrow$ **0.82**, kept ✓

Same within one class in crowds (pedestrians, parked cars).

One threshold, one car

τ=0.3: **1 box**
τ=0.5: **1 box**
τ=0.7: **1 box**
τ=0.9: **16 boxes**

low τ: merges neighbours
high τ: keeps duplicates

- **Crowds:** real objects may overlap strongly.
- **Thresholds:** one τ does not fit every class and scale.
- **Score myopia:** suppression ignores image evidence.
- **Discrete step:** NMS is not learned end to end.

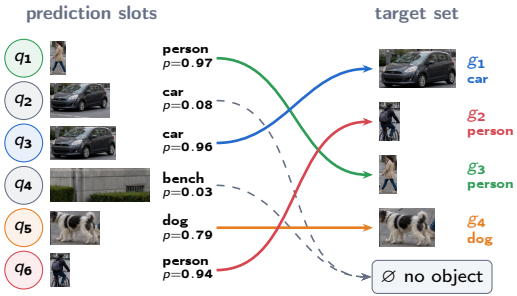
A softer alternative

Soft-NMS decays neighboring scores instead of deleting boxes:

$s_j \leftarrow s_j \, e^{-\mathrm{IoU}^2/\sigma}$.

Bodla et al., Soft-NMS, ICCV 2017. Boxes: Faster R-CNN and RetinaNet (COCO) on this scene.

Modern detection: set prediction



6 slots shown (DETR: $N = 100$); each target gets exactly one slot, the rest learn "no object" ⇒ no duplicates to suppress

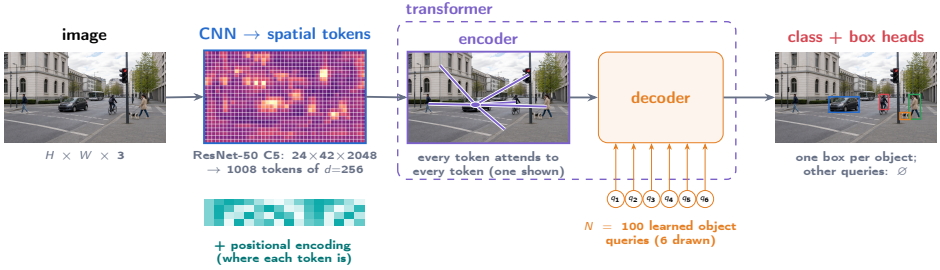
- Predict a fixed-size unordered set.
- Match each object to exactly one slot.
- Train unmatched slots as "no object."
- Duplicate removal is part of the learning objective.

no anchors **no NMS**

Here q_2 is a duplicate of the car and q_4 is background: both are matched to ∅. Slots are real detector boxes standing in for DETR outputs.

Carion et al., DETR, ECCV 2020.

DETR overview



- Global attention connects distant image regions.
- Learned queries compete for objects in parallel.
- Bipartite set loss trains the complete pipeline.
- Original DETR converges slowly and struggles with small objects.

Carion et al., DETR, ECCV 2020. Real input and C5 map; attention and outputs illustrative.

Hungarian matching

rows: prediction slots q_i; columns: targets g_j

green: optimal one-to-one match, total cost -2.8 (the minimum over all 360 assignments); q_2, $q_4 \to \varnothing$

Pairwise matching cost

$$\mathcal{C}(i,j) = -p_i(c_j) + \lambda_1 \|b_i - b_j\|_1 + \lambda_g \mathcal{L}_{\mathrm{GIoU}}.$$

Example, q_3 vs. g_1 (car), λ_1=5, λ_g=2:
$-0.96 + 5 \cdot 0.009 + 2 \cdot 0.06 = -0.80$

$$\hat{\sigma} = \arg\min_{\sigma} \sum_{j} \mathcal{C}(\sigma(j), j).$$

Match first; then compute class and box losses on the selected pairs.

Kuhn, 1955; Carion et al., DETR, 2020. Slots: RetinaNet boxes standing in for DETR outputs.

Detection datasets

Benchmark	Classes	Distinctive supervision	What it stresses
PASCAL VOC	20	boxes, compact taxonomy	foundational AP and transfer
COCO	80 things	boxes, masks, keypoints	small objects and crowded scenes
LVIS	1,203	instance masks, long tail	rare categories and calibration
Open Images	600 box classes	hierarchy and image-level labels	scale and label relationships

Dataset choice is a model choice

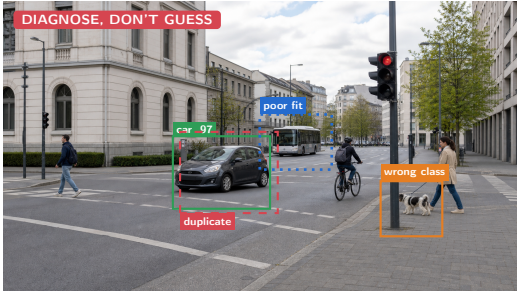
Object vocabulary, image source, annotation policy, and class frequency define what "good detection" means.

Benchmark scores do not transfer automatically

Audit domain shift, missing labels, duplicated categories, and deployment costs.

Everingham et al., 2010; Lin et al., 2014; Gupta et al., 2019; Kuznetsova et al., 2020.

Detection failure modes

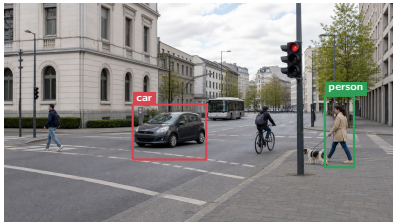


- **Classification:** wrong label or background hallucination.
- **Localization:** shifted or oversized box.
- **Coverage:** small or occluded objects missed.
- **Multiplicity:** duplicates or merged instances.
- **Shift:** sensor, weather, and geography differ.

Slice errors by class, scale, occlusion, and domain; one mAP number hides the cause.

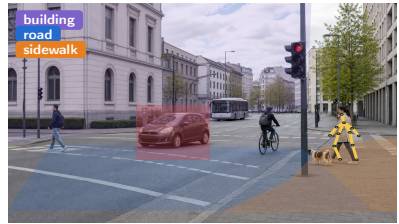
Boxes are coarse; pixels matter.

DETECTION



Boxes include background and erase shape.

DENSE PREDICTION



Masks preserve boundaries, shape, and contact.

Next: predict a label—or an instance identity—for every pixel.

He et al., Mask R-CNN, ICCV 2017. Base scene generated for this lecture; overlays are original TikZ.

Segmentation

Semantic segmentation

DENSE PREDICTION

Assign one semantic class to **every pixel**:

$$z = f_\theta(x) \in \mathbb{R}^{H \times W \times C},$$
$$\hat{y}_{ij} = \arg\max_c z_{ijc}.$$

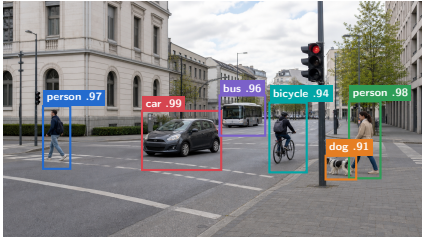
- Output remains aligned with the input image.
- "Stuff" (road, sky) and "things" (car, person) share one label map.
- Same-class objects are *not* separated into instances.

Classification asks *what*; segmentation asks *what at each location*.

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation," CVPR 2015.

Detection vs semantic segmentation

Detection: sparse objects



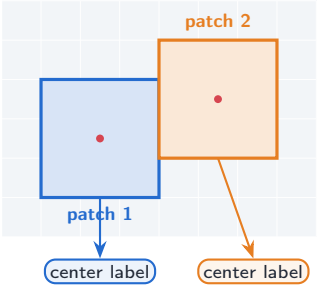
Output: a set $\{(c_k, s_k, b_k)\}_{k=1}^{K}$

Segmentation: dense labels

Output: a map $\hat{Y} \in \{1, \ldots, C\}^{H \times W}$

	Detection	Semantic segmentation
Geometry	coarse box extent	object boundaries and background
Identity	separates detections	merges pixels of the same class
Supervision	boxes	pixel-level masks

Naive patch classification



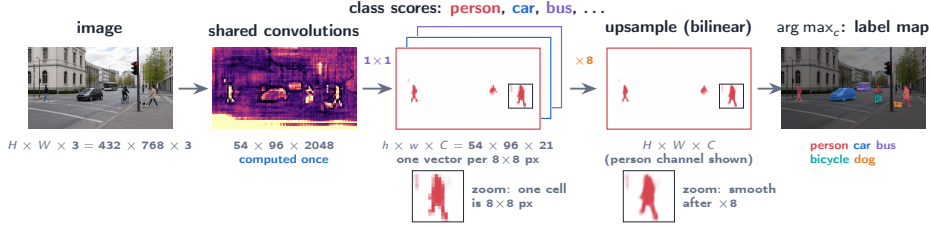
Train a classifier on a crop around each target pixel.

- Neighboring patches overlap almost completely.
- Their convolutional features are recomputed again and again.
- Small patches miss context; large patches multiply the cost.
- A full image needs roughly $H \times W$ classifier calls.

Bottleneck: correct idea, wasteful implementation—shared computation is missing.

Farabet et al., "Learning Hierarchical Features for Scene Labeling," IEEE TPAMI 2013.

Fully convolutional networks



Key conversion

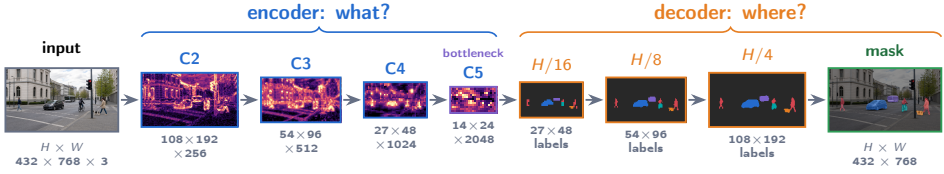
Replace dense classifier layers by convolutions. Spatial score maps emerge in one forward pass.

- accepts variable image sizes;
- shares features across locations;
- trades resolution for receptive field.

- Dense prediction does not require dense repetition.

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation," CVPR 2015.

Encoder–decoder design

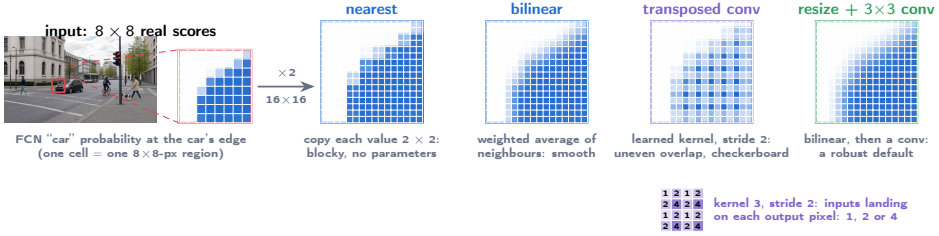


Downsampling enlarges the receptive field and builds invariance, but discards precise location.

Upsampling restores spatial resolution, but must reconstruct boundaries from compressed features.

Segmentation needs both global context and pixel-level localization.

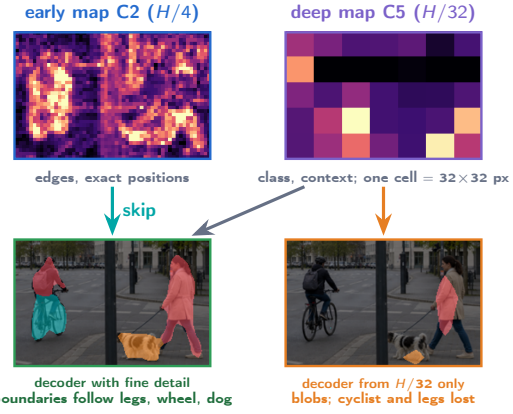
Upsampling methods



Upsampling increases sampling density; it cannot recover detail that the encoder never preserved.

Dumoulin and Visin, "A Guide to Convolution Arithmetic for Deep Learning," 2016; Odena et al., "Deconvolution and Checkerboard Artifacts," 2016.

Skip connections recover detail

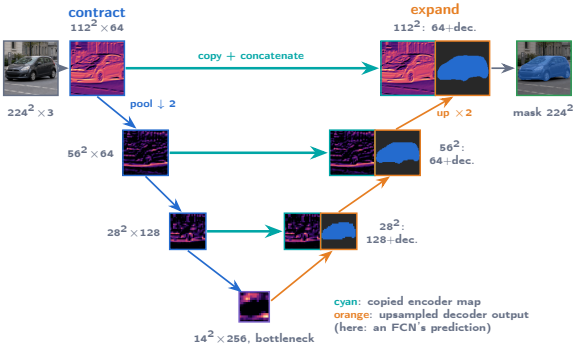


- Early maps retain edges, corners, and accurate coordinates.
- Deep maps encode class and broad context.
- Concatenation or addition lets the decoder use both.
- Feature sizes must be aligned by padding, cropping, or resizing.

Contrast with ResNet: these links bridge scales; residual links mainly ease optimization within a scale.

Long et al., CVPR 2015; Ronneberger, Fischer, and Brox, MICCAI 2015.

U-Net



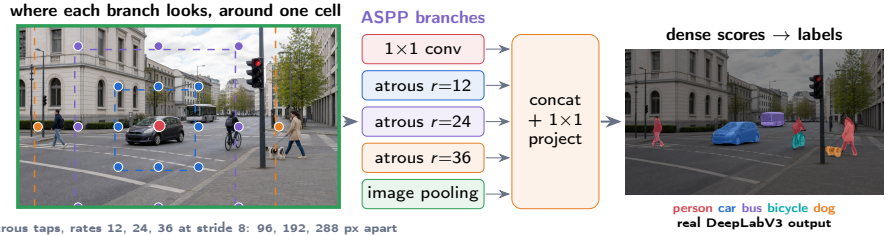
BIOMEDICAL ORIGIN

- Symmetric encoder and decoder.
- Skip features are **concatenated**, not merely added.
- Strong localization with limited annotated data.
- Modern U-Nets often add padding, normalization, and residual blocks.

Enduring idea: pair semantics with same-scale detail.

Ronneberger, Fischer, and Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI 2015.

DeepLab: context at multiple scales



Atrous convolution inserts gaps into the kernel:

$$y[p] = \sum_k w[k]\, x[p + rk].$$

- Larger field of view without another pooling step.
- Parallel rates capture objects at different scales.
- A light decoder can sharpen boundaries.

Multi-scale context disambiguates local texture using the surrounding scene.

Chen et al., "Rethinking Atrous Convolution for Semantic Image Segmentation," 2017; Chen et al., "Encoder–Decoder with Atrous Separable Convolution," ECCV 2018.

Segmentation losses

Pixel-wise cross-entropy

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{|\Omega|} \sum_{i \in \Omega} \sum_{c=1}^{C} w_c \, y_{ic} \log p_{ic}$$

Probabilistic and stable; class weights can protect rare labels.

Soft Dice loss

$$\mathcal{L}_{\mathrm{Dice}} = 1 - \frac{2 \sum_i p_i y_i + \varepsilon}{\sum_i p_i + \sum_i y_i + \varepsilon}$$

Directly rewards overlap; useful for small foreground regions.

Worked example: a rare class

reference y dog = 5.5% of pixels | model p(dog) (DeepLabV3) | per-pixel CE of the model

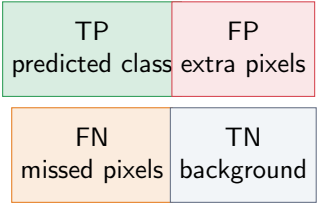
prediction	$\mathcal{L}_{\mathrm{CE}}$	$\mathcal{L}_{\mathrm{Dice}}$
model (above)	0.034	0.21
"always background" (p=0.05)	0.212	**0.95**

CE averages over many easy background pixels; Dice sees the dog region as a whole.

Common recipe: start with CE; add Dice, focal, or boundary terms only for a diagnosed imbalance.

Milletari, Navab, and Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," 3DV 2016.

Segmentation metrics



per class, over all pixels

$$\text{pixel accuracy} = \frac{\sum_c \text{TP}_c}{|\Omega|},$$

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c},$$

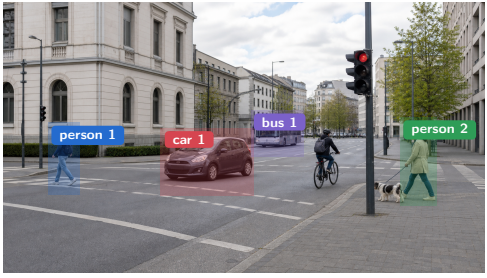
$$\text{mIoU} = \frac{1}{C} \sum_{c=1}^{C} \text{IoU}_c.$$

- Accuracy can look excellent when background dominates.
- mIoU gives each class equal weight.
- Fix ignore regions and resize policy before comparing models.

Check: a model predicts only "road." Which metric exposes the failure?

Everingham et al., "The PASCAL Visual Object Classes Challenge," IJCV 2010.

Instance segmentation



Predict a set of object-level masks:

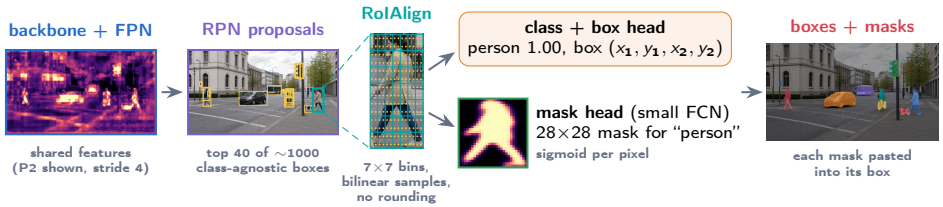
$$\{(c_k, s_k, b_k, M_k)\}_{k=1}^{K}.$$

- Each countable **thing** receives an identity.
- Overlapping objects remain separate.
- Masks capture shape; boxes still help localize proposals.
- "Stuff" is usually left to a semantic branch.

Instance segmentation predicts classes *and individuals*.

Hariharan et al., "Simultaneous Detection and Segmentation," ECCV 2014.

Mask R-CNN



One extra branch

Faster R-CNN supplies proposals, classes, and boxes; a parallel FCN predicts an $m \times m$ mask for each RoI.

- RoIAlign avoids coordinate quantization.
- Mask loss is pixel-wise binary cross-entropy.
- Detection and masks share backbone features.

Precise alignment matters when the output is a boundary, not just a box.

He et al., "Mask R-CNN," ICCV 2017.

Pose estimation and tracking

Pose estimation

Predict a structured set of landmarks for a person or object:

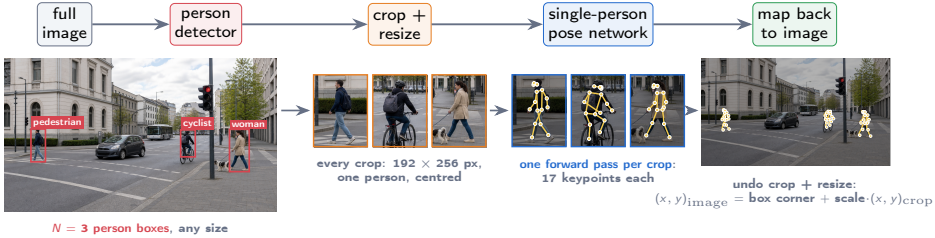
$$\mathcal{K} = \{(x_j, y_j, v_j)\}_{j=1}^{J},$$

where v_j records visibility or confidence.

- **2D / 3D pose:** keypoints in image coordinates or joints in camera/world coordinates.
- A fixed skeleton graph connects related joints.
- Multiple people add a second problem: assigning joints to identities.

Toshev and Szegedy, "DeepPose: Human Pose Estimation via Deep Neural Networks," CVPR 2014.

Top-down pose estimation



Why it works well

Each crop has normalized scale, little background, and one main subject; a high-resolution head can refine joint locations.

Detector bottleneck

A missed or poorly cropped person cannot be recovered. Runtime grows roughly with the number of detected people.

Top-down methods trade detector dependence and per-person cost for clean single-person reasoning.

Xiao, Wu, and Wei, "Simple Baselines for Human Pose Estimation and Tracking," ECCV 2018.

Bottom-up pose estimation

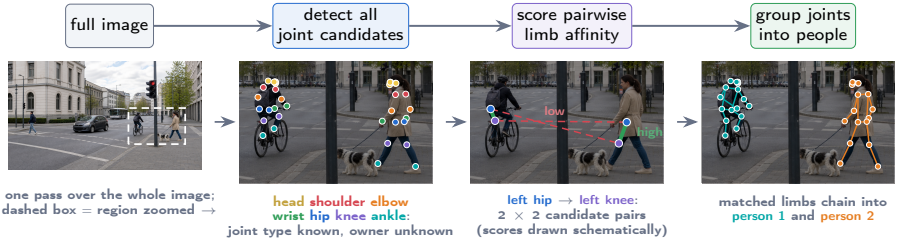


Image-level prediction

- Compute keypoint heatmaps once for the scene.
- Predict evidence that two joints belong to one limb.
- Assemble compatible limbs into person instances.

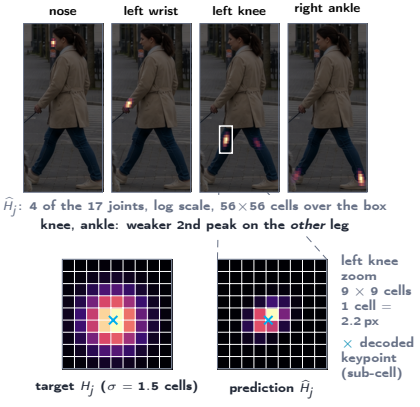
Trade-offs

- Cost depends weakly on crowd size.
- Small people share the same feature-map resolution.
- Grouping becomes ambiguous under overlap and occlusion.

Bottom-up methods detect body parts first and solve person identity through grouping.

Cao et al., "Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields," CVPR 2017 (OpenPose).

Heatmap regression



Encode the target joint $\boldsymbol{p}_j$ as a Gaussian bump:

$$H_j(\boldsymbol{p}) = \exp\left(-\frac{\|\boldsymbol{p} - \boldsymbol{p}_j\|_2^2}{2\sigma^2}\right),$$

$$\hat{\boldsymbol{p}}_j = \arg\max_{\boldsymbol{p}} \widehat{H}_j(\boldsymbol{p}).$$

- Train with mean-squared error or a normalized spatial loss; heatmaps preserve uncertainty better than direct (x, y) regression.
- Downsampling introduces quantization; refinement or soft-argmax restores subpixel precision.
- Multiple peaks can represent ambiguity or multiple people.

Newell, Yang, and Deng, "Stacked Hourglass Networks for Human Pose Estimation," ECCV 2016.

Pose metrics

PCK / PCKh

A joint is correct when

$$\|\hat{\boldsymbol{p}}_j - \boldsymbol{p}_j\|_2 \le \alpha L,$$

where L is a person, torso, or head scale. Average the correctness indicator over joints.

Object Keypoint Similarity

Let $\mathcal{V} = \{j : v_j > 0\}$ be the visible keypoints. Then

$$\mathrm{OKS} = \frac{1}{|\mathcal{V}|} \sum_{j \in \mathcal{V}} \exp\left(-\frac{d_j^2}{2s^2\kappa_j^2} \right).$$

Here d_j is keypoint error, s is person scale, and κ_j is keypoint tolerance.

Keypoint AP

Treat OKS like IoU: match poses at several thresholds, build precision–recall curves, then average AP.

COCO: AP@[.50:.95]

Compare pose scores only under the same scale, visibility, matching, and threshold protocol.

Andriluka et al., "2D Human Pose Estimation: New Benchmark and State of the Art Analysis," CVPR 2014; Lin et al., "Microsoft COCO," ECCV 2014.

Pose applications and failure modes

Applications

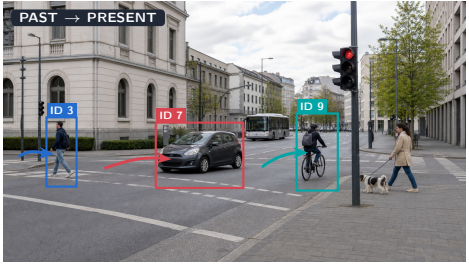
Sports analysis, human–robot interaction, ergonomics, animation/AR, rehabilitation, and gesture interfaces.

Failure modes

- occlusion, truncation, and crowded overlap;
- rare viewpoints and unusual articulation;
- blur, low resolution, or loose clothing;
- left/right swaps and implausible limbs;
- domain and demographic shift.

Ronchi and Perona, "Benchmarking and Error Diagnosis in Multi-Instance Pose Estimation," ICCV 2017.

Multi-object tracking



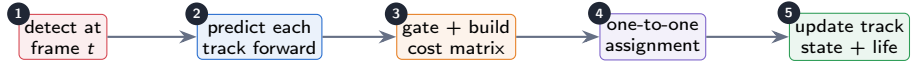
Given detections over video, estimate trajectories

$$\mathcal{T}_i = \{(t, b_t^i, c_i, \mathrm{ID}_i)\}_{t=t_s}^{t_e}.$$

- Localize each object at each time.
- Preserve a stable identity through motion and occlusion.
- Start and terminate tracks as objects enter or leave.
- Diagnose false/missed tracks, fragmentation, and **ID switches**.

Milan et al., "MOT16: A Benchmark for Multi-Object Tracking," arXiv 2016.

Tracking by detection



① detections d_j at frame t (**white boxes, Faster R-CNN**)
② tracks T_i predicted from $t-1$ (**dashed**; $t-1$ **simulated**)

	d1	d2	d3	d4	d5
T1	0.14	×	×	×	×
T2	×	0.12	×	×	×
T3	×	×	0.15	×	×
T4	×	×	×	0.19	×
T5	×	×	×	×	×

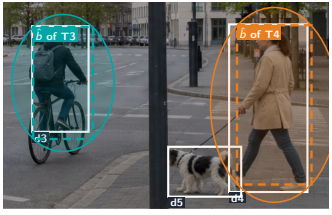
③ cost $c_{ij} = 1 - \mathrm{IoU}(\hat{b}_i, d_j)$; × = **gated** (IoU < 0.3)
④ ◯ **min-cost one-to-one matches; T5 (behind the pole) and d5 (the dog) stay unmatched**

⑤ **update (lower image): matched → correct the predicted state with its detection, refresh confidence + appearance; unmatched detection → tentative new track; unmatched track → keep briefly, delete after repeated misses**

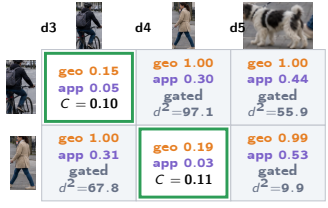
A simple online loop becomes a tracker when association and lifecycle rules are explicit.

Bewley et al., "Simple Online and Realtime Tracking," ICIP 2016 (SORT).

Motion, geometry, and appearance for association



motion: ellipse = gate around the Kalman prediction; geometry: dashed $\hat{b}_i$ vs white d_j



appearance: $1 - \cos$ of ResNet-18 embeddings of the crops (stored track look vs detection)

geo $= c_{ij}^{\text{geo}} = 1 - \text{IoU}$

app $= c_{ij}^{\text{app}} = 1 - \cos$

$C = 0.5\,\text{geo} + 0.5\,\text{app}$

grey: outside the motion gate ($d^2 > 5.99$), never matched

Hungarian match

T4 vs d5: woman vs dog, app 0.53

A constant-velocity Kalman model predicts box center, scale, velocity, and covariance:

$$\boldsymbol{x}_t = F\boldsymbol{x}_{t-1} + \boldsymbol{w}_t, \qquad \hat{\boldsymbol{x}}_{t|t-1} = F\hat{\boldsymbol{x}}_{t-1|t-1}.$$

For track i and detection j:

$$c_{ij}^{\text{geo}} = 1 - \text{IoU}(\hat{b}_i, d_j), \qquad c_{ij}^{\text{app}} = 1 - \boldsymbol{e}_i^{\top}\boldsymbol{e}_j,$$

with normalized appearance embeddings.

Combine valid pairs as

$$C_{ij} = \lambda_g c_{ij}^{\text{geo}} + \lambda_a c_{ij}^{\text{app}},$$
$$\pi^* = \arg\min_{\pi} \sum_i C_{i,\pi(i)}.$$

Motion rejects implausible jumps; IoU favors spatial continuity; appearance helps after crossings and short occlusions.

Bewley et al., SORT, ICIP 2016; Wojke et al., Deep SORT, ICIP 2017; Kuhn, Hungarian method, 1955.

Main takeaways

1. Learn useful representations

CNNs encode locality and translation structure; ViTs use content-dependent interaction. Both can serve as reusable backbones.

2. Match the output to the task

Classification predicts labels; detection boxes; segmentation pixels; pose keypoints; tracking identities through time.

3. Evaluate what matters

Choose task-aligned metrics and inspect errors by scale, class, occlusion, localization, and identity.

4. Build by diagnosing failures

Start with a measurable baseline, visualize its mistakes, and add complexity only to address a specific failure.

Align representation, output, loss, metric, and data with the failure you need to fix.

Thank you!

Jhony H. Giraldo: jhony.giraldo@telecom-paris.fr
(https://jhonygiraldo.github.io/)